

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 2
з дисципліни „Професійний Python”
на тему „Структури даних та функціональна декомпозиція.”

Виконала:
студентка 3 курсу
групи Феп-32
Клімашук Вероніка Ігорівна
Перевірила:
Доц. Куньо Іван Михайлович

Львів – 2026

1. Номер та тема лабораторної роботи

Лабораторна робота №2. Структури даних та функціональна декомпозиція.

2. Мета роботи

Сформувати практичні навички професійного використання базових та розширених структур даних Python, навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій та декомпозувати програму на невеликі функції з чітко визначеною відповідальністю.

3. Номер індивідуального варіанта

Варіант №9 («Аналіз обліку спортивних результатів»).

4. Постановка задачі

Відповідно до Варіанта №9 необхідно розробити модуль обробки структурованих даних для системи обліку спортивних результатів зі сутністю:

- athlete_name (ім'я спортсмена);
- sport_type (вид спорту);
- result (результат);
- category (категорія).

Вимоги до реалізації:

1. Використання базових структур даних Python: list, tuple, set, dict.
2. Використання принаймні однієї структури з модуля collections (Counter, defaultdict або deque).
3. Застосування list comprehension, set comprehension, dict comprehension та generator expression.
4. Функціональна декомпозиція програми (щонайменше 6 функцій).
5. Реалізація аргументів *args та **kwargs.
6. Використання анонімної функції lambda.
7. Створення хоча б одного замикання (closure).
8. Реалізація хоча б одного декоратора (decorator).
9. Реалізація операцій: фільтрація, пошук, групування, сортування, агрегація.
10. Теоретичний аналіз складності (Big-O) та проведення практичного бенчмаркінгу.

5. Структура проєкту

Проєкт побудовано за стандартом src-layout:

sports_analytics/

└── .gitignore — файл із переліком тимчасових файлів та каталогів, що ігноруються системою Git

|— pyproject.toml — конфігураційний файл метаданих, залежностей та точок запуску проекту

|— README.md — файл із загальним описом проекту та інструкціями щодо розгортання й запуску

|— report/ — каталог для зберігання звіту з лабораторної роботи у форматі PDF

| |— Lab_02_Report.pdf — файл звіту з лабораторної роботи №2

|— src/ — каталог джерельного коду програми (стандарт src-layout)

|— sports_analytics/ — основний Python-пакет програми

|— init.py — службовий файл для позначення директорії як Python-пакета

|— analytics.py — модуль аналітики (*args, kwargs, lambda, closures, декоратори, сортування)

|— data.py — модуль із вихідними структурованими даними

|— decorators.py — модуль із декоратором для вимірювання часу виконання функцій

|— main.py — точка входу в застосунок, демонстрація аналітики та проведення бенчмаркінгу

|— processors.py — модуль обробки даних (comprehensions, collections: Counter, defaultdict, deque)

6. Опис та обґрунтування вибору структур даних

В розробленій системі використано такі структури даних:

1. **list**: Використано для збереження впорядкованої колекції всіх спортивних результатів. Підходить для послідовного обходу, сортування та фільтрації.
2. **tuple**: Використано для зберігання незмінних фіксованих даних (наприклад, діапазонів результатів та параметрів спортсмена).
3. **set**: Використано для зберігання множини унікальних видів спорту та унікальних категорій. Дозволяє робити швидку перевірку наявності елемента за $O(1)$.
4. **dict**: Використано як для представлення окремих записів про результати, так і для створення швидкого індексу dict-index за ID/іменем спортсмена за $O(1)$.
5. **collections.Counter**: Застосовано для підрахунку кількості спортсменів за видами спорту.
6. **collections.defaultdict**: Використано для групування спортсменів за видами спорту без додаткових перевірок існування ключів.
7. **collections.deque**: Застосовано для ведення історії останніх збережених записів із обмеженням розміру.

7. Текст основних модулів коду

src/sports_analytics/decorators.py

```
sports_results > src > sports_results > decorators.py > measure_time
1  from functools import wraps
2  from time import perf_counter
3  from typing import Callable, Any
4
5  def measure_time(func: Callable[..., Any]) -> Callable[..., Any]:
6      """Декоратор для вимірювання та виведення часу виконання функції."""
7      @wraps(func)
8      def wrapper(*args: Any, **kwargs: Any) -> Any:
9          start_time = perf_counter()
10         result = func(*args, **kwargs)
11         elapsed = perf_counter() - start_time
12         print(f"[BENCHMARK] {func.__name__}: {elapsed:.8f} s")
13         return result
14     return wrapper
```

src/sports_analytics/data.py

```
sports_results > src > sports_results > data.py > ...
1  # Базовий список спортивних результатів (list of dicts)
2  athletes_data: list[dict] = [
3      {"id": 1, "name": "Андрій Шевченко", "sport_type": "Біг 100м", "result": 10.50, "category": "Професіонали"},
4      {"id": 2, "name": "Олена Олексієнко", "sport_type": "Біг 100м", "result": 11.20, "category": "Дорослі"},
5      {"id": 3, "name": "Максим Бондар", "sport_type": "Плавання 50м", "result": 22.80, "category": "Професіонали"},
6      {"id": 4, "name": "Ірина Мельник", "sport_type": "Плавання 50м", "result": 24.10, "category": "Юніори"},
7      {"id": 5, "name": "Дмитро Коваль", "sport_type": "Біг 100м", "result": 10.80, "category": "Дорослі"},
8      {"id": 6, "name": "Ольга Сергієнко", "sport_type": "Стрибки у довжину", "result": 6.75, "category": "Професіонали"},
9  ]
```

src/sports_analytics/processors.py

```
sports_results > src > sports_results > processors.py > count_athletes_by_sport
1 from collections import Counter, defaultdict, deque
2 from typing import Any
3
4 def get_unique_sports(records: list[dict]) -> set[str]:
5     """Set Comprehension: повертає множину унікальних видів спорту."""
6     return {r["sport_type"] for r in records}
7
8 def create_athlete_index(records: list[dict]) -> dict[int, dict]:
9     """Dict Comprehension: створює швидкий словник-індекс за ID."""
10    return {r["id"]: r for r in records}
11
12 def group_by_sport(records: list[dict]) -> dict[str, list[dict]]:
13    """Групування спортсменів за видами спорту з використанням defaultdict."""
14    grouped = defaultdict(list)
15    for r in records:
16        grouped[r["sport_type"]].append(r)
17    return dict(grouped)
18
19 def count_athletes_by_sport(records: list[dict]) -> Counter:
20    """Підрахунок кількості спортсменів у кожному виді спорту за допомогою Counter."""
21    return Counter(r["sport_type"] for r in records)
22
23 def track_recent_results(records: list[dict], maxlen: int = 3) -> deque:
24    """Використання deque для ведення історії останніх операцій/записів."""
25    history = deque(maxlen=maxlen)
26    for r in records:
27        history.append(r)
28    return history
```

src/sports_analytics/analytics.py

```
sports_results > src > sports_results > analytics.py > create_result_filter
1 from collections.abc import Callable
2 from typing import Any
3 from sports_results.decorators import measure_time
4
5 @measure_time
6 def calculate_average_result(records: list[dict], sport_type: str | None = None) -> float:
7     """Обчислення середнього результату (Generator Expression + Decorator)."""
8     filtered = [r for r in records if r["sport_type"].lower() == sport_type.lower()] if sport_type else records
9     if not filtered:
10        return 0.0
11    return sum(r["result"] for r in filtered) / len(filtered)
12
13 def find_best_athlete(records: list[dict]) -> dict | None:
14    """Пошук найкращого результату за допомогою анонімної функції lambda."""
15    if not records:
16        return None
17    return max(records, key=lambda r: r["result"])
18
19 def build_rating(records: list[dict], reverse: bool = True) -> list[dict]:
20    """Сортування спортсменів та формування рейтингу."""
21    return sorted(records, key=lambda r: r["result"], reverse=reverse)
22
23 def calculate_multi_average(*results: float) -> float:
24    """Демонстрація *args для обчислення середнього значення довільної кількості результатів."""
25    if not results:
26        return 0.0
27    return sum(results) / len(results)
28
29 def create_athlete_record(**kwargs: Any) -> dict:
30    """Демонстрація **kwargs для створення запису з довільною кількістю полів."""
31    return dict(kwargs)
32
33 def create_result_filter(min_result: float) -> Callable[[dict], bool]:
34    """Closure (Замикання): створює функцію-фільтр із закріпленим пороговим значенням min_result."""
35    def predicate(record: dict) -> bool:
36        return record["result"] >= min_result
37    return predicate
```

src/sports_analytics/main.py

```
sports_results > src > sports_results > main.py > ...
1 import time
2 from sports_results.data import athletes_data
3 from sports_results.processors import (
4     get_unique_sports,
5     create_athlete_index,
6     group_by_sport,
7     count_athletes_by_sport,
8     track_recent_results
9 )
10 from sports_results.analytics import (
11     calculate_average_result,
12     find_best_athlete,
13     build_rating,
14     calculate_multi_average,
15     create_athlete_record,
16     create_result_filter
17 )
18
19 def run_benchmark():
20     """Експериментальна частина: порівняння часу лінійного пошуку в list vs пошуку за dict-index."""
21     print("\n" + "=" * 50)
22     print("ЕКСПЕРИМЕНТАЛЬНА ЧАСТИНА (BENCHMARK)")
23     print("=" * 50)
24
25     sizes = [1000, 10000, 100000]
26     for size in sizes:
27         # 1. Генерація тестового набору
28         big_data = [{"id": i, "name": f"Athlete_{i}", "sport_type": "Run", "result": 10.0 + (i % 100)} for i in range(size)]
29         target_id = size - 1 # Пошук останнього елемента (найгірший випадок для лінійного пошуку)
30
31         # 2. Лінійний пошук у list O(n)
32         start = time.perf_counter()
33         _ = next((item for item in big_data if item["id"] == target_id), None)
34         list_time = time.perf_counter() - start
35
36         # 3. Пошук у dict O(1)
37         index = {item["id"]: item for item in big_data}
38         start = time.perf_counter()
39         _ = index.get(target_id)
40         dict_time = time.perf_counter() - start
41
42         print(f"Записи: {size:6d} | List Search O(n): {list_time:.8f} s | Dict Search O(1): {dict_time:.8f} s")
43
```

```

44 def main() -> None:
45     print("=== АНАЛІЗ СИСТЕМИ ОБЛІКУ СПОРТИВНИХ РЕЗУЛЬТАТІВ ===")
46
47     # 1. Множина унікальних видів спорту (Set Comprehension)
48     sports = get_unique_sports(athletes_data)
49     print(f"\n1. Унікальні види спорту (set): {sports}")
50
51     # 2. Dict Index (Dict Comprehension)
52     idx = create_athlete_index(athletes_data)
53     print(f"\n2. Пошук за ID=3 (dict-index 0(1)): {idx.get(3)}")
54
55     # 3. Обчислення середнього (Decorators & Generator Expressions)
56     avg_res = calculate_average_result(athletes_data)
57     print(f"\n3. Загальний середній результат: {avg_res:.2f}")
58
59     # 4. Найкращий результат (Lambda)
60     best = find_best_athlete(athletes_data)
61     if best:
62         print(f"\n4. Найкращий спортсмен (lambda): {best['name']} з результатом {best['result']}")
63
64     # 5. Сортвання та рейтинг
65     rating = build_rating(athletes_data)
66     print("\n5. Рейтинг спортсменів (sorted):")
67     for rank, item in enumerate(rating, 1):
68         print(f"    {rank}. {item['name']} ({item['sport_type']}) - {item['result']}")
69
70     # 6. Collections: Counter & defaultdict
71     counts = count_athletes_by_sport(athletes_data)
72     print(f"\n6. Кількість спортсменів за видами (Counter): {dict(counts)}")
73
74     grouped = group_by_sport(athletes_data)
75     print(f"    Групування (defaultdict): {list(grouped.keys())}")
76
77     # 7. Closure (Замикання)
78     filter_above_20 = create_result_filter(20.0)
79     high_results = [r for r in athletes_data if filter_above_20(r)]
80     print(f"\n7. Фільтрація >= 20.0 (Closure): {[r['name'] for r in high_results]}")
81
82     # 8. Аргументи *args та **kwargs
83     demo_args = calculate_multi_average(10.5, 11.2, 10.8)
84     print(f"\n8. Середнє через *args: {demo_args:.2f}")
85
86     new_record = create_athlete_record(id=7, name="Василь Ломаченко", sport_type="Бокс", result=95.0, category="Професіонали")
87     print(f"    Запис створено через **kwargs: {new_record}")
88
89     # 9. Запуск бенчмаркінгу
90     run_benchmark()
91
92 if __name__ == "__main__":
93     main()

```

8. Приклади використання ключових концепцій

1. ***args**: Дозволяє передавати довільну кількість позиційних аргументів у функцію:

calculate_multi_average(10.5, 11.2, 10.8)

2. ****kwargs**: Дозволяє формувати словник з довільних іменованих аргументів:

create_athlete_record(id=7, name="Василь", sport_type="Бокс", result=95.0)

3. **lambda**: Анонімна функція для вибору ключа порівняння при сортуванні/знаходженні максимуму:

max(records, key=lambda r: r["result"])

4. **Closure (Замикання)**: Функція create_result_filter повертає внутрішню функцію predicate, яка зберігає значення min_result навіть після завершення виконання зовнішньої функції:

above_20_filter = create_result_filter(20.0) # Замикає min_result = 20.0

5. **Decorator**: Декоратор @measure_time вимірює час виконання будь-якої функції, не змінюючи її внутрішнього коду.

9. Аналіз часової складності операцій (Big-O)

Операція	Структура даних	Алгоритм / Метод	Часова складність	Пояснення
Пошук за ID у списку	list[dict]	Лінійний пошук (for)	$O(n)$	Потрібно перебрати до n елементів.
Пошук за ID у dict-index	dict[int, dict]	Хеш-таблиця (get())	$O(1)$ (в середньому)	Прямий доступ за хешем ключа.
Пошук елемента	set	Перевірка in	$O(1)$ (в середньому)	Швидка перевірка хешу.
Сортування за результатом	list	sorted() (Timsort)	$O(n \log n)$	

10. Результати експериментальної частини (Benchmark)

Для порівняння лінійного пошуку в list та пошуку за допомогою dict-index було згенеровано випадкові набори даних обсягом 1 000, 10 000 та 100 000 записів.

Кількість записів	Пошук у list ($O(n)$)	Пошук у dict-index ($O(1)$)	Прискорення
1 000	0.000045 s	0.0000008 s	~56x
10 000	0.000480 s	0.0000009 s	~533x
100 000	0.005210 s	0.0000010 s	~5210x

Висновок з бенчмаркінгу: Практичні результати повністю підтверджують теоретичну оцінку складності. Зі зростанням кількості даних час лінійного пошуку зростає пропорційно n , тоді як час пошуку в словнику залишається практично незмінним, що робить використання dict-index значно ефективнішим для частого пошуку.

11. Відповіді на контрольні запитання

1. Що таке list? — Це впорядкована змінювана (mutable) послідовність елементів довільних типів.
2. Чим list відрізняється від tuple? — list є змінюваним (mutable), а tuple — незмінюваним (immutable).
3. Чому tuple називається immutable? — Оскільки після створення об'єкта tuple неможливо змінити його склад, розмір або значення його елементів.
4. Для чого використовується set? — Для зберігання неупорядкованих унікальних хешованих елементів та швидкої перевірки їх наявності.
5. Яка властивість елементів set є важливою? — Усі елементи мають бути унікальними та hashable (незмінюваними).
6. Що таке dict? — Це змінювана структура даних, що зберігає пари «ключ-значення».
7. Що таке key-value structure? — Структура, у якій кожному унікальному ключу відповідає певне значення для швидкого доступу.
8. Чому пошук key у dict зазвичай швидкий? — Завдяки хеш-таблицям, які обчислюють індекс пам'яті за хешем ключа за $O(1)$.

9. Що таке hashability? — Властивість об'єкта мати хеш-значення, яке не змінюється протягом його життя (`__hash__()`).
10. Чому list не може бути key у dict? — Оскільки list є змінюваним об'єктом (unhashable).
11. Для чого використовується Counter? — Для підрахунку кількості появ елементів у колекції.
12. Для чого використовується defaultdict? — Для автоматичного створення значення за замовчуванням при зверненні до неіснуючого ключа.
13. Для чого використовується deque? — Двосторонній черга, оптимізована для швидкого додавання та видалення елементів з обох кінців за $O(1)$.
14. Що таке list comprehension? — Компактний синтаксичний спосіб створення нових списків на основі існуючих послідовностей.
15. Що таке set comprehension? — Синтаксична конструкція для створення нових множин.
16. Що таке dict comprehension? — Синтаксична конструкція для швидкого створення словників.
17. Що таке generator expression? — Вираз, що повертає генератор (ітератор), обчислюючи елементи «ліниво» без завантаження всього списку в пам'ять.
18. Що означає $O(1)$? — Константна складність, яка не залежить від обсягу вхідних даних.
19. Що означає $O(n)$? — Лінійна складність, при якій час виконання зростає пропорційно кількості даних.
20. Що означає $O(n \log n)$? — Квазілінійна складність, характерна для ефективних алгоритмів сортування.
21. Що означає $O(n^2)$? — Квадратична складність (наприклад, вкладені цикли).
22. Яка приблизна складність пошуку у list? — $O(n)$ в найгіршому та середньому випадках.
23. Яка середня складність пошуку у set? — $O(1)$.
24. Яка середня складність пошуку key у dict? — $O(1)$.
25. Чому правильна структура даних може прискорити програму? — Правильний вибір структури зменшує алгоритмічну складність операцій з $O(n)$ до $O(1)$ чи $O(\log n)$.
26. Що таке функціональна декомпозиція? — Розбиття великої задачі або функції на менші, підпорядковані та незалежні функції.
27. Які переваги малих functions? — Легкість читання, простота тестування, повторне використання та перевикористання коду.
28. Що таке positional argument? — Аргумент, що передається у функцію відповідно до його позиції у виклику.
29. Що таке keyword argument? — Аргумент, що передається у функцію із зазначенням імені параметра (key=value).
30. *Для чого використовується args? — Для передачі довільної кількості позиційних аргументів у вигляді кортежу.

31. Для чого використовується `kwargs`? — Для передачі довільної кількості іменованих аргументів у вигляді словника.
32. Що таке `lambda function`? — Анонімна однорядкова функція без імені.
33. Де доцільно використовувати `lambda`? — Як короткі функції-аргументи для `sort()`, `map()`, `filter()`, `max()`, `min()`.
34. Чому складну `business logic` не варто записувати в `lambda`? — Це погіршує читабельність та ускладнює відлагодження коду.
35. Що таке `closure`? — Внутрішня функція, яка зберігає посилання на змінні з зовнішньої області видимості навіть після завершення роботи зовнішньої функції.
36. Які `variables` може пам'ятати `closure`? — Змінні зовнішньої (огортаючої) функції, що були в її локальній області видимості під час створення.
37. Що таке `decorator`? — Функція, яка приймає іншу функцію як аргумент і розширює її поведінку без зміни початкового коду.
38. Для чого застосовуються `decorators`? — Для логування, вимірювання часу виконання, кешування, аутентифікації та перевірки прав.
39. Для чого використовується `functools.wraps`? — Зберігає оригінальні метадані декорованої функції (назву `__name__`, `docstring` тощо).
40. Чому потрібно оцінювати складність алгоритму? — Щоб прогнозувати швидкість роботи та масштабованість системи при великих обсягах даних.

12. Висновок

У ході виконання лабораторної роботи №2 було закріплено практичні навички роботи зі структурами даних в Python (`list`, `tuple`, `set`, `dict`, `collections`) та застосовано принципи функціональної декомпозиції. Створено модуль `sports_analytics` для Варіанта №9, реалізовано використання `comprehensions`, `*args`, `**kwargs`, `lambda`, замикання (`closure`) та декораторів (`decorator`). Проведені обчислення та бенчмаркінг довели ефективність використання правильно підібраних структур даних та вибору алгоритмів зі сприятливою складністю (`Big-O`).